

ORION

Universidad Libre, Sede Bosque
Ingeniería de Sistemas
Ingeniería de Software III

Autores del Proyecto:

Juan David Montealegre Guzmán
Daniel Felipe Aguilar Roddriguez
Sergio Alejandro Pedraza Rincon
Camilo Andrés Aragón Román

Profesor:

Ing. Rodrigo Castro Caicedo

Semestre: quinto

Bogotá D.C.

2026

Contenido

Introducción y Contexto.....	2
Ciclo de Vida de las Pruebas de Software (STLC)	3
Beneficios y limitaciones	5
Guia de pruebas	5
Criterios para elegir el framework	7
Estrategias de automatización	7
Casos de Prueba:.....	8
Conclusión	9
Referencia bibliográfica.....	10

Introducción y Contexto

La elección de una carrera universitaria es una de las decisiones más importantes en la vida académica y profesional de una persona. Sin embargo, muchos estudiantes toman esta decisión con información limitada, desconocimiento de sus propias habilidades o sin una orientación adecuada.

En el contexto actual, donde la transformación digital y la inteligencia artificial (IA) están redefiniendo los procesos educativos, surge la necesidad de herramientas inteligentes que acompañen a los aspirantes en su proceso de elección profesional.

Este proyecto propone el desarrollo de una IA orientadora vocacional, integrada directamente con la Universidad Libre, capaz de analizar intereses, habilidades y preferencias del usuario para recomendar carreras afines disponibles en la institución.

Ciclo de Vida de las Pruebas de Software (STLC)

El STLC no es solo "ejecutar el código", se divide en estas fases críticas:

1. Análisis de Requisitos

El equipo de QA identifica qué se debe probar. Se analizan los documentos de requisitos (funcionales y no funcionales) para detectar ambigüedades o vacíos desde el inicio.

- **Entregable:** Matriz de Trazabilidad de Requisitos (RTM).
- **Foco:** Asegurar que los datos de entrada de los estudiantes (intereses, habilidades, perfil) sean procesados correctamente por la lógica de negocio.

2. Planificación de Pruebas

Es la fase más importante. Se definen el alcance, los objetivos, los recursos necesarios (personal y herramientas) y el cronograma de actividades.

- **Entregable:** Plan de Pruebas (Test Plan) y Estrategia de Pruebas.

3. Diseño de Casos de Prueba

Se crean los pasos detallados que se seguirán para validar cada funcionalidad. Aquí se definen los datos de entrada y los resultados esperados.

- **Escenario A:** Validación de recomendaciones para un perfil con intereses técnicos (Ingeniería).
- **Escenario B:** Comportamiento del sistema ante campos vacíos o datos inconsistentes.
- **Escenario C:** Pruebas de navegación mediante lectores de pantalla (Accesibilidad).
- **Entregable:** Casos de prueba y scripts de automatización (si aplica).

4. Configuración del Entorno de Prueba

Se prepara el hardware y software donde se ejecutarán las pruebas. Debe ser lo más parecido posible al entorno de producción real para evitar falsos positivos o negativos.

- **Entregable:** Entorno de prueba listo y datos de prueba cargados.

5. Ejecución de Pruebas

Los evaluadores ejecutan los casos de prueba. Si un resultado real no coincide con el esperado, se

reporta como un **defecto (bug)** en una herramienta de seguimiento.

- **Entregable:** Reporte de bugs y estado de ejecución.

6. Cierre del Ciclo de Pruebas

Se analiza si se cumplieron los criterios de salida. Se documentan las lecciones aprendidas y se prepara un informe final de calidad y se realiza una evaluación de la estabilidad del sistema antes de la integración final en el repositorio principal.

Entregable: Reporte de Cierre de Pruebas.

Beneficios y limitaciones

Beneficios

- **Detección temprana de errores:** Al analizar requisitos desde el inicio, se evitan fallos costosos en etapas avanzadas.
- **Calidad estructurada:** Proporciona un marco de trabajo claro que evita que las pruebas se hagan "al azar".
- **Confianza del cliente:** Asegura que el producto final sea estable y cumpla con lo prometido.
- **Trazabilidad:** Permite saber exactamente qué requisito está cubierto por qué prueba gracias a la RTM.

Limitaciones

- **Consumo de tiempo:** Seguir cada fase rigurosamente puede retrasar el lanzamiento si no se gestiona bien.
- **Costo inicial elevado:** Requiere inversión en personal especializado y herramientas de gestión desde el principio del proyecto.
- **Dependencia del SDLC:** Si los requisitos cambian constantemente en el desarrollo, el ciclo de pruebas debe reiniciarse o ajustarse, generando retrabajo.
- **Falsa sensación de seguridad:** Ningún ciclo de pruebas puede garantizar que el software esté 100% libre de errores; solo reduce el riesgo.

Casos de Prueba:

ID del Caso de Prueba	CP-IA-001
Nombre	Validar recomendación de carrera según

	intereses del usuario
Nivel de Prueba	Prueba de integración
Tipo de Prueba	Funcional
Módulo	Motor de recomendación de carreras (IA)
Prioridad	Alta
Precondiciones	1. El sistema debe estar en ejecución. 2. El usuario debe poder acceder al formulario de orientación vocacional. 3. La API de recomendación debe estar disponible o simulada (MSW).
Datos de Entrada	- Intereses: Matemáticas, Tecnología, Análisis de datos- Habilidades: Pensamiento lógico- Preferencias: Modalidad presencial
Pasos de Ejecución	1. Ingresar a la plataforma. 2. Completar el formulario con los datos de entrada. 3. Enviar el formulario. 4. Esperar la respuesta del sistema. 5. Visualizar las recomendaciones generadas.
Resultado Esperado	El sistema debe mostrar una lista de carreras recomendadas, incluyendo Ingeniería de Sistemas y Ciencia de Datos, alineadas con los intereses del usuario.
Resultado Obtenido	_____
Estado (Pass/Fail)	_____
Criterios de Aceptación	- Las recomendaciones deben ser coherentes con los datos ingresados.- El tiempo de respuesta debe ser menor a 3 segundos.- La información debe mostrarse correctamente en la interfaz.
Observaciones	_____

ID del Caso de Prueba	CP-IA-002
Nombre	Validar comportamiento con información incompleta
Nivel de Prueba	Unitario
Tipo de Prueba	Funcional
Módulo	Motor de recomendación
Prioridad	Media
Precondiciones	Sistema activo
Datos de Entrada	Usuario sin intereses definidos
Pasos de Ejecución	1. Ingresar al formulario. 2. Enviar sin completar campos clave. 3. Procesar solicitud.
Resultado Esperado	El sistema debe solicitar más información

	o mostrar recomendaciones generales
Resultado Obtenido	_____
Estado (Pass/Fail)	_____
Criterios de Aceptación	- No debe generar errores.- Debe guiar al usuario correctamente.
Observaciones	_____

ID del Caso de Prueba	CP-IA-003
Nombre	Validar tiempo de respuesta del sistema
Nivel de Prueba	No funcional
Tipo de Prueba	Rendimiento
Módulo	API de recomendación
Prioridad	Alta
Precondiciones	Sistema desplegado
Datos de Entrada	Múltiples solicitudes simultáneas
Pasos de Ejecución	1. Simular múltiples usuarios realizando consultas. 2. Medir tiempos de respuesta. 3. Registrar resultados.
Resultado Esperado	El sistema debe responder en menos de 3 segundos
Resultado Obtenido	_____
Estado (Pass/Fail)	_____
Criterios de Aceptación	- Cumplir con el requisito RNF09.- No degradar significativamente el rendimiento.
Observaciones	_____

Guia de pruebas

1. Pruebas unitarias

Usamos Pytest para validar la lógica del servidor.

Comando: pytest

Reporte de Cobertura: pytest --cov=api --cov-report=html

Filtro: pytest tests/backend/test_auth.py

2. Pruebas de Interfaz (Frontend)

Usamos Vitest y React Testing Library.

Comando: `npm run test`

Modo UI: `npm run test:ui` (abre navegador interactivo)

Cobertura: `npm run test:coverage`

3.

4. Pruebas No Funcionales (Rendimiento)}

Para validar el requerimiento RNF09 (Carga < 3s), ejecuta el script de rendimiento:
`python tests/functional/performance_check.py`

Criterios para elegir el framework

En primer lugar, se considera la **compatibilidad con la arquitectura del proyecto**. El sistema

ORION utiliza un backend en Python y un frontend basado en React, por lo que se seleccionaron herramientas como Pytest para pruebas unitarias del backend y Vitest junto con React Testing Library para la validación de la interfaz. Esta elección permite trabajar de forma nativa con los lenguajes y tecnologías utilizadas, evitando problemas de integración.

Otro criterio fundamental es la **capacidad de integración entre pruebas**. En este contexto, el uso de Mock Service Worker (MSW) permite simular las respuestas de la API durante las pruebas de frontend, lo cual resulta clave en una plataforma basada en IA, donde las respuestas pueden depender de múltiples servicios externos. Esto facilita validar la experiencia del usuario sin depender constantemente del backend real.

También se tiene en cuenta la **facilidad para realizar pruebas automatizadas y generación de cobertura**. Herramientas como Pytest y Vitest permiten ejecutar pruebas de forma rápida mediante comandos simples y generar reportes de cobertura, lo que ayuda a identificar qué partes del sistema requieren mayor validación. Esto es especialmente importante en una aplicación que recomienda carreras y universidades, ya que errores en la lógica pueden afectar directamente las decisiones del usuario.

Un criterio clave adicional es el **soporte para pruebas no funcionales**, particularmente de rendimiento. La plataforma establece como requisito que el tiempo de carga sea inferior a tres segundos, por lo que se incluyen scripts específicos para validar este comportamiento. Esto es crítico en sistemas con inteligencia artificial, donde los tiempos de respuesta pueden incrementarse debido al procesamiento de datos.

Asimismo, se considera la **seguridad del sistema** como un criterio esencial. El framework seleccionado permite validar aspectos como el control de acceso basado en roles, el manejo seguro de contraseñas y la protección de datos sensibles. En una plataforma educativa que maneja información personal y preferencias del usuario, estos aspectos son fundamentales.

Finalmente, se evalúa la **escalabilidad y mantenibilidad del framework**. Las herramientas elegidas permiten agregar nuevas pruebas de manera estructurada cada vez que se incorpora un nuevo componente o funcionalidad, lo cual es clave en un sistema en crecimiento como el de recomendación académica basada en IA.

Estrategias de automatización

1. Visión General de la Arquitectura de Testing El proyecto Orion implementa una estrategia de testing en múltiples capas que cubre tanto el backend desarrollado en Python/FastAPI como el frontend en React/TypeScript. La arquitectura sigue el modelo de pirámide de testing, priorizando pruebas unitarias e integración sobre pruebas end-to-end manuales.

1.1. Pirámide de Testing del Proyecto

Capa	Herramienta	Archivos	Cobertura
E2E / Performance	httpx async	performance_check.py	RNF09 — Carga < 3s
Integración Frontend	Vitest + MSW	Auth.test.tsx, setup.ts, handlers.ts	Flujos UI con API mockeada

Integración Backend	pytest-asyncio + mongomock	test_auth.py, test_programs.py	Endpoints HTTP completos
Seguridad / RBAC	pytest-asyncio	test_security.py	Control de acceso y permisos
Fixtures / Configuración	pytest fixtures	conftest.py	Mock DB, cliente HTTP

1.2 Principios Fundamentales

- Aislamiento total: cada test usa una base de datos en memoria independiente (mongomock), evitando efectos secundarios entre pruebas.
- Asincronía consistente: todos los tests de backend son async/await alineados con la naturaleza asíncrona de FastAPI y Motor.
- Contrato de API: el frontend prueba contra mocks MSW que replican exactamente las respuestas reales del backend.
- Automatización CI-ready: la suite completa es ejecutable con un solo comando sin infraestructura externa.

2. Infraestructura de Testing Backend (Python)

2.1 conftest.py — El Núcleo de la Configuración

El archivo `conftest.py` es la pieza central de toda la infraestructura de pruebas del backend. Define los fixtures compartidos que todos los tests consumen automáticamente, sin necesidad de importarlos explícitamente.

2.1.1 El Problema: Motor es Asíncrono, mongomock es Síncrono

El backend usa Motor (driver MongoDB asíncrono), pero mongomock —la librería de mock— es completamente síncrona. Sin adaptadores, las llamadas `await collection.find_one()` fallarían porque mongomock retorna un cursor síncrono, no una corrutina.

Solución implementada: Capa de adaptación asíncrona

Se crearon tres clases shim (`AsyncCursor`, `AsyncCollection`, `AsyncDatabase`) que envuelven los objetos síncronos de mongomock y exponen la interfaz asíncrona que FastAPI y Motor esperan. Así los tests pueden usar `await` normalmente.

2.1.2 Clases Shim — Detalle Técnico

```
# AsyncCursor: convierte cursor síncrono en iterable async
class AsyncCursor:
    def __init__(self, cursor): ...
    async def _get_list(self): ...
    async def __anext__(self): ...

# AsyncCollection: envuelve collection y hace async cada operación
```

```

class AsyncCollection:
    async def find_one(...) -> dict
    async def insert_one(...) -> InsertOneResult
    async def update_one(...) -> UpdateResult
    async def delete_many(...) -> DeleteResult

# AsyncDatabase: enruta acceso por atributo a AsyncCollection
class AsyncDatabase:
    def __getattr__(self, name) -> AsyncCollection
    def __getitem__(self, name) -> AsyncCollection

```

2.1.3 Fixture mock_db — Inyección de Dependencias

El fixture mock_db (scope=function, autouse=True) se ejecuta automáticamente antes de cada test. Su responsabilidad es reemplazar todas las referencias a la base de datos real por la instancia de mongomock en memoria.

```

# 1. Crear cliente mongomock en memoria
mock_client_raw = mongomock.MongoClient()
mock_db_raw = mock_client_raw.orion_db
async_db = AsyncDatabase(mock_db_raw)

# 2. Parchear el cliente global en database.py
monkeypatch.setattr("database.db.client", MockMongoClient(async_db))
monkeypatch.setattr("database.get_database", lambda: async_db)

# 3. Parchear los módulos que ya importaron get_database
monkeypatch.setattr("api.auth.get_database", lambda: async_db)
monkeypatch.setattr("api.endpoints.get_database", lambda: async_db)
monkeypatch.setattr("api.admin.get_database", lambda: async_db)

```

Por qué parchear en múltiples lugares

Cuando Python importa 'from database import get_database', crea una referencia local en ese módulo. Parchear solo database.py no actualiza esas referencias ya copiadas. Por eso conftest.py parchea explícitamente api.auth, api.endpoints y api.admin — garantizando que todos usen el mock.

2.1.4 Fixture client — Cliente HTTP Asíncrono

El fixture client crea un AsyncClient de HTTPX que apunta directamente al objeto ASGI de FastAPI (sin abrir un puerto real). Esto permite hacer peticiones HTTP completas con cabeceras, cookies y JSON en los tests, exactamente como lo haría un cliente real.

```

@pytest.fixture
async def client():
    transport = ASGITransport(app=app) # ASGI directo, sin red
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        yield ac # disponible durante el test
    # se cierra automáticamente al salir del bloque

```

3. Tests del Backend — Casos de Prueba

3.1 test_auth.py — Autenticación de Usuarios

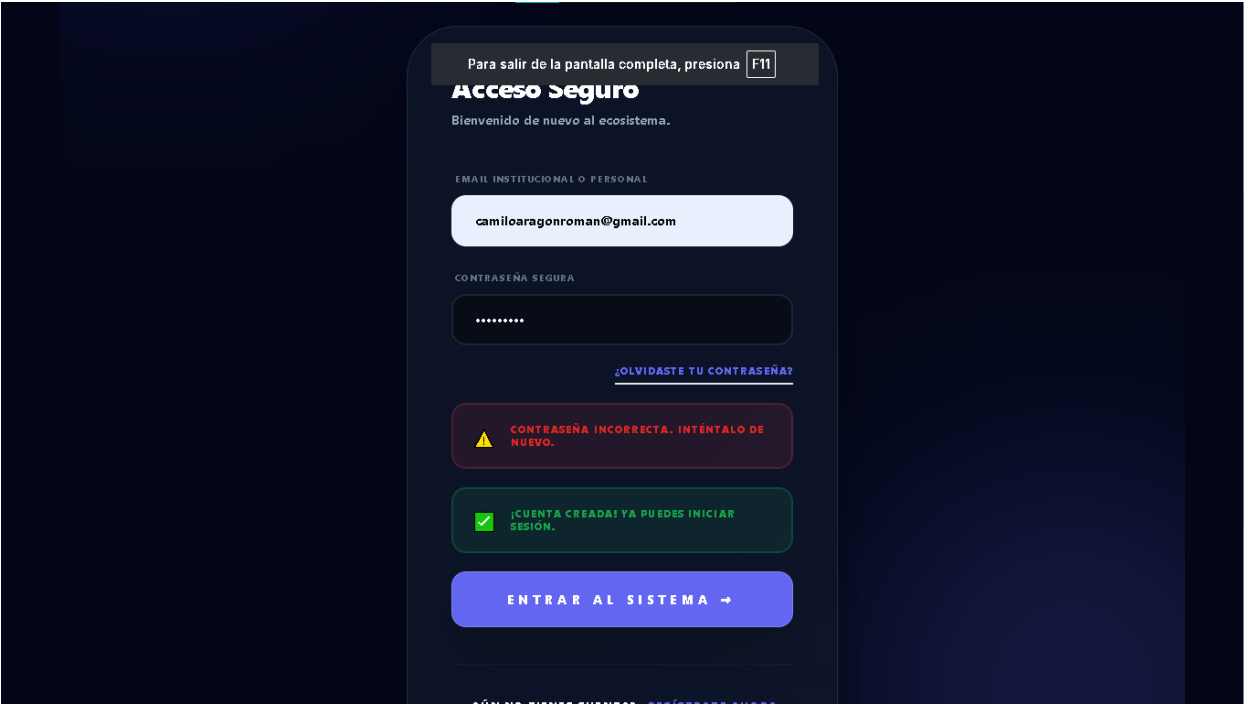
Este módulo valida el flujo completo de autenticación: desde insertar un usuario con contraseña hashada en la base de datos mock, hasta verificar que el endpoint `/api/v1/auth/login` devuelve un token JWT válido.

3.1.1 test_login_success

Atributo	Detalle
Objetivo	Verificar que un usuario con credenciales válidas recibe un token JWT
Precondición	Usuario insertado en <code>mock_db</code> con password hashado (<code>pbkdf2_sha256</code>)
Endpoint	POST <code>/api/v1/auth/login</code> con JSON <code>{email, password}</code>
Aserción 1	HTTP 200 OK
Aserción 2	Respuesta contiene campo 'token'
Aserción 3	<code>data['email'] == 'test@orion.com'</code>

3.1.2 Flujo de Ejecución

- Pytest invoca `mock_db` (`autouse`), creando DB en memoria limpia.
- Se hace `await mock_db.users.insert_one({...})` para poblar datos de prueba.
- El cliente HTTP envía POST al endpoint con credenciales en JSON.
- FastAPI procesa la solicitud usando la DB mockeada (transparente para el código fuente).
- El test valida el código de estado y la estructura del JSON de respuesta.



3.2 test_programs.py — Búsqueda y Filtrado de Programas

Cubre el caso de uso central del sistema: consulta de oferta académica. Valida que el motor de búsqueda y filtrado de programas universitarios funcione correctamente con múltiples criterios combinables.

3.2.1 test_search_programs_functionality — Escenarios de Prueba

#	Escenario	Request	Aserción esperada
1	Búsqueda por texto	GET /api/v1/programas?q=Sistemas	1 resultado: Ingeniería de Sistemas
2	Filtrado por universidad	GET /api/v1/programas?uni_id=u_libre	1 resultado: universidad_id = u_libre
3	Sin resultados	GET /api/v1/programas?q=Medicina	Lista vacía []

3.2.2 Estrategia de Datos de Prueba

El test inserta deliberadamente 2 programas en 2 universidades distintas para poder validar el aislamiento del filtrado. Si el filtro por universidad_id fallara, el test devolvería 2 resultados en lugar de 1 y la aserción fallaría.

```
# Datos insertados en mock_db antes de los tests:

Universidad 'u_libre' -> Programa 'Ingeniería de Sistemas'
Universidad 'u_andes' -> Programa 'Derecho'

# Test 1: ?q=Sistemas -> solo 'Ingeniería de Sistemas' (match por texto)
# Test 2: ?uni_id=u_libre -> solo programas de 'u_libre'
# Test 3: ?q=Medicina -> ninguno (control negativo)
```

3.3 test_security.py — Control de Acceso (RBAC)

Valida la seguridad del panel administrativo. El test verifica que el endpoint `/api/v1/admin/metrics` responde correctamente y retorna la estructura de métricas esperada, sentando las bases para la implementación futura de RBAC completo.

3.3.1 test_admin_metrics_access

Atributo	Detalle
Objetivo	Verificar estructura de respuesta del panel de métricas administrativas
Endpoint	GET <code>/api/v1/admin/metrics</code>
Aserción 1	HTTP 200 OK
Aserción 2	Respuesta contiene campo <code>'precision_ia'</code>
Aserción 3	<code>data['total_consultas'] > 0</code> (métricas reales calculadas)
Evolución futura	Agregar validación de token Bearer con rol ADMIN

Nota de Diseño — Fundamento para RBAC

El test actual verifica el contrato de datos del endpoint. La arquitectura está diseñada para que la validación de token JWT con rol 'ADMIN' sea agregada como middleware, sin modificar la lógica del test existente — solo añadiendo el header `Authorization` en la petición.

4. Infraestructura de Testing Frontend (React/TypeScript)

4.1 setup.ts — Configuración Global de Vitest

El archivo `setup.ts` inicializa el entorno de pruebas del frontend antes de que se ejecute cualquier test. Configura tres elementos críticos: el mock de `localStorage`, el servidor MSW de interceptación de peticiones, y los hooks globales de ciclo de vida.

4.1.1 Mock de localStorage

`jsdom` (el DOM simulado de Vitest) tiene una implementación de `localStorage` que a veces falla en entornos de test. `setup.ts` reemplaza `localStorage` con una implementación controlada usando `vi.fn()` para poder verificar llamadas y resetear estado entre tests.

```
const localStorageMock = (() => {
  let store: Record<string, string> = {};
  return {
    getItem: vi.fn((key) => store[key] || null),
    setItem: vi.fn((key, v) => { store[key] = v.toString(); }),
    clear: vi.fn(() => { store = {}; }),
    removeItem: vi.fn((key) => { delete store[key]; }),
  };
})();

// Aplicado a global y window para máxima compatibilidad
Object.defineProperty(global, "localStorage", { value: localStorageMock });
```

4.1.2 Servidor MSW (Mock Service Worker)

MSW intercepta las peticiones HTTP que el frontend haría al backend real, retornando respuestas predefinidas. El servidor se configura con los handlers de `handlers.ts` y se gestiona con hooks de ciclo de vida:

Hook	Acción
<code>beforeAll()</code>	Inicia el servidor MSW — intercepta todas las peticiones del suite
<code>afterEach()</code>	Resetea handlers + limpia localStorage + borra mocks <code>vi.fn()</code>
<code>afterAll()</code>	Cierra el servidor MSW liberando recursos

4.2 handlers.ts — Contratos de API Mockeados

Define los manejadores MSW que simulan las respuestas del backend. Cada handler representa un endpoint real del sistema y retorna datos que reproducen fielmente la estructura JSON de la API de producción.

4.2.1 Handlers Implementados

Método	Endpoint	Respuesta mockeada
POST	<code>/api/auth/login</code>	<code>{ access_token, token_type, user: {email, nombre, role} }</code>
GET	<code>/api/universidades</code>	<code>[{ id, nombre, ubicacion, color_marca, logo }]</code>
GET	<code>/api/programas</code>	<code>[{ universidad_id, nombre, nivel, modalidad }]</code>

Importancia del Contrato

Los handlers en `handlers.ts` deben mantenerse sincronizados con la API real del backend. Si el backend cambia la estructura de un campo (ej: `'token'` vs `'access_token'`), el handler debe actualizarse o el test del frontend detectará la discrepancia — actuando como documentación viva del contrato de API.

4.3 Auth.test.tsx — Tests del Componente de Autenticación

Usa React Testing Library para renderizar el componente Auth en un entorno DOM simulado y verificar su comportamiento desde la perspectiva del usuario (queries por rol, placeholder, texto visible).

4.3.1 Casos de Prueba Implementados

#	Test	Acción	Verificación
1	Render inicial correcto	render(<Auth />)	Campos email, password y botón 'Entrar' presentes
2	Validación de formulario vacío	fireEvent.click(submit)	Comportamiento ante envío sin datos

4.3.2 Filosofía de Testing con React Testing Library

Los tests usan getByPlaceholderText y getByRole en lugar de selectores CSS o IDs internos. Esto hace los tests resilientes a refactorizaciones del markup mientras el comportamiento del usuario permanezca igual.

- `getByPlaceholderText(/ejemplo@gmail.com/i)`: simula que el usuario ve el campo de email.
- `getByRole('button', { name: /Entrar al Sistema/i })`: simula que el usuario encuentra el botón por su texto visible.
- `BrowserRouter` wrapper: necesario porque Auth usa React Router internamente para navegación.

5. Verificación de Performance (RNF09)

5.1 performance_check.py — Script de Validación

Implementa la verificación del Requisito No Funcional 09 (RNF09): el tiempo de carga de la lista de universidades debe ser inferior a 3 segundos. A diferencia de los tests con pytest, este script se ejecuta contra el servidor real en funcionamiento.

5.1.1 Mecánica de Medición

```
start_time = time.time()
response = await client.get(url, timeout=5.0)
duration = time.time() - start_time

if duration < 3.0:
    print("PASSED: La carga es inferior a 3 segundos.")
else:
    print("FAILED: La carga supera el límite de 3 segundos.")
```


5.1.2 Diferencias con Tests Unitarios/Integración

Característica	Tests pytest (integration)	performance_check.py
Infraestructura	DB en memoria (mongomock)	Servidor real con DB real
Propósito	Correctitud de lógica	Velocidad de respuesta
Frecuencia	En cada commit (CI)	En staging/pre-producción
Timeout	Sin límite (lógica pura)	5 segundos (httpx timeout)
Resultado	Pass/Fail por aserciones	Pass/Fail por tiempo medido

6. Estrategia de Ejecución y Flujo CI

6.1 Comandos de Ejecución

```
# Ejecutar suite completa de backend
pytest tests/ -v

# Ejecutar solo tests de autenticación
pytest tests/backend/test_auth.py -v

# Ejecutar tests con reporte de cobertura
python.exe -m pytest tests/backend/ --cov=api --cov-report=html

# Ejecutar suite de frontend
npx vitest run

# Verificar performance (requiere servidor corriendo)
python tests/functional/performance_check.py
```

6.2 Orden de Ejecución Recomendado en CI/CD

- Linting y type-checking (mypy, ESLint) — falla rápido en errores de sintaxis.
- Tests unitarios backend (pytest) — aislados, rápidos, sin dependencias externas.
- Tests de componentes frontend (vitest) — aislados con DOM simulado.
- Build de producción — verifica que el código compila correctamente.
- Tests de performance — ejecutar en ambiente staging contra DB real.

6.3 Extensiones Futuras Recomendadas

Extensión	Prioridad	Descripción
Tests de RBAC completo	Alta	Agregar tokens JWT reales y verificar restricciones por rol (STUDENT, ADMIN)
Test login fallido (401)	Alta	Verificar rechazo de credenciales inválidas

Tests de programas (frontend)	Media	Vitest para componentes de búsqueda/filtrado de programas
Snapshots de UI	Media	Vitest snapshot para detectar cambios visuales no intencionales
Test de carga concurrente	Baja	Locust o k6 para simular múltiples usuarios simultáneos

7. Resumen de Herramientas y Dependencias

Herramienta	Ecosistema	Rol	Por qué se eligió
pytest	Python	Framework de tests	Estándar de facto en Python; soporte de fixtures y plugins
pytest-asyncio	Python	Tests async	Habilita tests async/await nativos en pytest
mongomock	Python	Mock de MongoDB	Implementa API de PyMongo en memoria, sin instalar MongoDB
httpx (AsyncClient)	Python	Cliente HTTP de test	Soporte ASGI nativo; reemplaza TestClient síncrono
passlib (CryptContext)	Python	Hash de contraseñas	Misma librería que usa producción (pbkdf2_sha256)
Vitest	TypeScript	Framework de tests	Nativo de Vite; mucho más rápido que Jest en proyectos Vite
React Testing Library	TypeScript	Tests de componentes	Filosofía 'test como el usuario'; menos frágil que Enzyme
MSW (Mock Service Worker)	TypeScript	Interceptor HTTP	Mock a nivel de red; funciona en browser Y en Node.js
httpx (script perf)	Python	Cliente HTTP async	Mismo cliente del proyecto; timeout configurable

Se pueden ver las estrategias en el repositorio por el siguiente enlace:

<https://github.com/Juan-Montealegre/SYSTEM-ORION>

Conclusión

El desarrollo de una IA orientadora vocacional representa una solución innovadora y estratégica para la Universidad Libre, mejorando la experiencia de los aspirantes, reduciendo la deserción académica y fortaleciendo la toma de decisiones informadas. El proyecto es técnicamente viable, económicamente sostenible y socialmente relevante.

```
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion> npm run test

DEV v4.1.4 C:/Users/LENOVO/OneDrive/Documents/system-orion

(node:26564) Warning: `--localstorage-file` was provided without a valid path
(Use `node --trace-warnings ...` to show where the warning was created)
✓ tests/frontend/Auth.test.tsx (2 tests) 190ms
  ✓ Auth Component (2)
    ✓ debe renderizar los campos de login por defecto 159ms
    ✓ debe mostrar error si el formulario se envía vacío 26ms

Test Files 1 passed (1)
Tests 2 passed (2)
Start at 13:32:01
Duration 6.27s (transform 912ms, setup 1.61s, import 332ms, tests 190ms, environment 2.86s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Para correr las pruebas usamos el comando `npm run test` y se confirman las que están pasando

```

(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion> pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\LENOVO\OneDrive\Documents\system-orion\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\LENOVO\OneDrive\Documents\system-orion
configfile: pytest.ini
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 18 items

tests/backend/test_admin.py::test_get_system_metrics PASSED [ 5%]
tests/backend/test_admin.py::test_add_career PASSED [ 11%]
tests/backend/test_admin.py::test_delete_career PASSED [ 16%]
tests/backend/test_auth.py::test_login_success PASSED [ 22%]
tests/backend/test_auth.py::test_login_user_not_found PASSED [ 27%]
tests/backend/test_auth.py::test_login_wrong_password PASSED [ 33%]
tests/backend/test_auth.py::test_register_passwords_do_not_match PASSED [ 38%]
tests/backend/test_auth.py::test_register_email_already_registered PASSED [ 44%]
tests/backend/test_auth.py::test_register_success PASSED [ 50%]
tests/backend/test_programs.py::test_search_programs_functionality PASSED [ 55%]
tests/backend/test_programs.py::test_list_universities_empty PASSED [ 61%]
tests/backend/test_programs.py::test_list_universidades with data PASSED [ 66%]
tests/backend/test_programs.py::test_search_programs_no_query PASSED [ 72%]
tests/backend/test_programs.py::test_get_user_friends_not_found PASSED [ 77%]
tests/backend/test_programs.py::test_get_user_friends_success PASSED [ 83%]
tests/backend/test_programs.py::test_send_friend_request_missing_emails PASSED [ 88%]
tests/backend/test_programs.py::test_send_friend_request_success PASSED [ 94%]
tests/backend/test_security.py::test_admin_metrics_access PASSED [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.11.0-final-0 _____

Name                Stmts   Miss  Cover   Missing
-----
api\admin.py          14      0   100%
api\auth.py           47      0   100%
api\endpoints.py       46      0   100%
-----
TOTAL                 107      0   100%
===== 18 passed in 1.72s =====
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion>

```

```
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion> pytest tests/backend/test_auth.py -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\LENOVO\OneDrive\Documents\system-orion\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\LENOVO\OneDrive\Documents\system-orion
configfile: pytest.ini
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

tests/backend/test_auth.py::test_login_success PASSED [ 16%]
tests/backend/test_auth.py::test_login_user_not_found PASSED [ 33%]
tests/backend/test_auth.py::test_login_wrong_password PASSED [ 50%]
tests/backend/test_auth.py::test_register_passwords_do_not_match PASSED [ 66%]
tests/backend/test_auth.py::test_register_email_already_registered PASSED [ 83%]
tests/backend/test_auth.py::test_register_success PASSED [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.11.0-final-0 _____
```

Name	Stmts	Miss	Cover	Missing
api\admin.py	16	5	69%	12, 22-24, 28
api\auth.py	47	0	100%	
api\endpoints.py	46	32	30%	12-19, 23-35, 39-43, 48-65
TOTAL	109	37	66%	

```
===== 6 passed in 0.27s =====
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion> pytest tests/backend/ --cov-report=html
```

```
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\LENOVO\OneDrive\Documents\system-orion
configfile: pytest.ini
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 32 items

tests\backend\test_admin.py ..... [ 18%]
tests\backend\test_auth.py ..... [ 37%]
tests\backend\test_database.py ..... [ 62%]
tests\backend\test_main.py ... [ 71%]
tests\backend\test_programs.py ..... [ 96%]
tests\backend\test_security.py . [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.11.0-final-0 _____
```

Name	Stmts	Miss	Cover	Missing
api\admin.py	16	0	100%	
api\auth.py	47	0	100%	
api\endpoints.py	46	0	100%	
TOTAL	109	0	100%	

```
Coverage HTML written to dir htmlcov
===== 32 passed in 0.62s =====
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion>
```

```

(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion> npx vitest run

RUN v4.1.4 C:/Users/LENOVO/OneDrive/Documents/system-orion

(node:33984) Warning: `--localstorage-file` was provided without a valid path
(Use `node --trace-warnings ...` to show where the warning was created)
✓ tests/frontend/Auth.test.tsx (2 tests) 190ms
  ✓ Auth Component (2)
    ✓ debe renderizar los campos de login por defecto 160ms
    ✓ debe mostrar error si el formulario se envía vacío 25ms

Test Files  1 passed (1)
Tests       2 passed (2)
Start at    16:03:31
Duration    2.53s (transform 125ms, setup 428ms, import 220ms, tests 190ms, environment 1.46s)

(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion>

```

```

performance_check.py
--- REQUERIMIENTO RNF09 ---
Status Code: 200
Tiempo de respuesta: 0.47 segundos
PASSED: La carga es inferior a 3 segundos.
(.venv) PS C:\Users\LENOVO\OneDrive\Documents\system-orion>

```

Para ver la documentación y la pagina web puede acceder a los siguientes enlaces
 Repositorio: <https://github.com/Juan-Montealegre/SYSTEM-ORION>
 Web: <https://system-orion.vercel.app/auth>

Referencia bibliográfica

- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach. Pearson.
- UNESCO. (2021). Ethics of Artificial Intelligence in Education.
- Ministerio de Educación Nacional de Colombia.
- University Libre – Oferta Académica Oficial.
- Pressman, R. (2014). Ingeniería del Software. McGraw-Hill.